RHAPSODY

CUSTOMER SECURITY ADVISORY

eMudhra CA compromise

Rhapsody Identity customer security advisory

PREPARED BY	DATE	AUDIENCE	VERSION
Rhapsody Security and Engineering	October 27, 2026	Customer use	1.0

eMudhra CA compromise

CONTENTS

01 Summary
02 Likelihood of exploitation
03 Is your deployment affected?
04 What Rhapsody is doing
05 What you can do now
06 References
A Certificate fingerprints

SECTION 01

Summary

On October 17, 2026, an independent security researcher disclosed that the private keys for eight intermediate certificate authority (CA) certificates operated by eMudhra (emSign) could be recovered from the public certificates. Anyone holding those keys can issue a TLS certificate for any hostname. Browsers blocked the intermediates within days, and eMudhra has revoked them.

Rhapsody Identity runs on a Java Development Kit (JDK) that is installed separately on each Identity server. That JDK keeps its own certificate trust store and its own list of blocked certificates (`blocked.certs`). Browser blocklists and operating system updates don't change either one, and Java does not automatically check certificate revocation. JDK vendors block compromised certificates by releasing JDK updates with an updated blocked list, and Amazon Corretto and Oracle have both released fixed builds for every Java version Identity uses (see section 5.1). Those updates have to be applied to the JDK itself, separately from operating system patching. Until the JDK is updated or the compromised certificates are blocked manually, an Identity server that validates a remote server's certificate against public CAs could accept a certificate forged with one of the compromised keys.

LIKELIHOOD

Low

Rhapsody assessment: the likelihood of exploitation against your Rhapsody Identity deployment is **low**. In typical deployments, Identity's main outbound connection is EMPI notification delivery, and the notification sender uses its own configured trust store rather than the JDK default. Exploitation also requires an attacker who already controls part of the network path between Identity and a remote endpoint.

Even so, Rhapsody recommends applying the appropriate JDK update on every Identity server. It is low risk, takes a few minutes per server plus a restart, and closes the gap for every connection the Identity JVM makes.

RHAPSODY CLOUD CUSTOMERS

Rhapsody is applying the update to all Rhapsody-hosted Identity environments. No action is required from you. Any restarts will be coordinated through scheduled maintenance windows.

CUSTOMER-HOSTED CUSTOMERS

The JDK on your Identity servers is managed by your organization. Follow the steps in section 5.

Patching Identity's JDK does not protect the systems that call Identity

For most customers the larger exposure is likely to be their own and third-party systems: integrations calling Identity, and connections between other systems generally. Any application whose runtime has its own bundled trust store (Java, Node.js, Python, Go and others) may be affected, because browser and operating system updates do not reach it. Review those systems with their owners, and contact the appropriate vendors for their fixes; section 3.3 covers the connections into Identity.

SECTION 02

Likelihood of exploitation

Rhapsody rates the likelihood of exploitation as **low**. For an attack to succeed against your deployment, all the following must be true at the same time:

1. **The connection trusts the affected public roots.** That means the JDK's default trust store (`cacerts`), or an Identity trust store that contains the emSign ECC Root CA - G3 or Sectigo AAA Certificate Services root. Connections that trust only specific partner certificates or a private CA are not exposed.
2. **The attacker is on the network path** between Identity and the remote endpoint, through compromised DNS, a compromised network segment, or a hostile proxy. Most Identity deployments run inside hospital networks or private cloud environments with private connectivity to downstream systems, which considerably lowers the chance of this occurring.
3. **The attacker has forged a certificate for that specific endpoint** using one of the eight recovered keys. This is technically simple but must be done per target.

Several factors reduce the risk further. Identity's notification sender, which carries most of its outbound traffic, uses its own configured trust store rather than `cacerts`. To be vulnerable, the compromised certificates would have had to be manually added to the trust store, or the notification sender's configuration changed to use `cacerts`.

What keeps the risk from being negligible: the keys have been recoverable since April 2025, the exposure remains until the JDK is updated, and an EMPI is a high-value target. Its outbound connections can carry patient demographic data and, for directory integrations, user credentials. The realistic threat is a targeted, well-positioned attacker, not opportunistic exploitation.

Regardless of the risk being low, Rhapsody recommends applying the fix on every server.

SECTION 03

Is your deployment affected?

Any connection where Identity is the TLS client and checks the remote server's certificate against the public CA chain is potentially affected. The table below lists the Identity connections to review. Whichever trust store a connection uses, the blocked-certificates list in the JDK update protects it, because the JDK applies the block list to every TLS connection it makes.

3.1 Identity connections

Connection	Trust store used	Exposure
Outbound EMPI notifications (JSON over HTTPS REST to downstream receivers)	The notification sender's own configured trust store	**Low.** Exposed only if the option to also use the JDK default trust store is enabled, or the configured trust store contains the affected roots.
Access Manager LDAPS connections to Active Directory or LDAP	`ad.truststorePath` or `opendj.truststorePath`. If not set, the JDK default trust store is used.	**Potentially affected** where no dedicated trust store is set, or where the directory certificate was imported into the JDK's `cacerts`. These connections carry user credentials.
Identity Augment functionality	Often the JDK default trust store	**Potentially affected**, if in use.
Custom integrations that call external APIs or data sources over HTTPS	Often the JDK default trust store	**Uncommon but potentially affected**, if any exist.
Other outbound TLS connections that use an Identity trust store (for example, OIDC single sign-on metadata retrieval)	The configured trust store, such as `truststore.jks` or `truststoreClient.jks`	**Affected only if** that trust store contains the emSign ECC Root CA - G3 or Sectigo AAA Certificate Services root.

To see whether an Identity trust store contains either root, list it with the JDK's `keytool` (for example, `keytool -list -v -keystore truststore.jks`) and look for "emSign" or "AAA Certificate Services" in the output. Appendix A lists the compromised certificates' SHA-256 fingerprints.

3.2 Not affected

- Plaintext connections, such as HL7 v2 over MLLP without TLS.
- Connections whose trust store holds only specific partner certificates or a private CA, including pinned mutual TLS.
- Inbound TLS where Identity is the server and does not validate client certificates.
- Users of the Identity web interface. Browsers have already blocked the compromised certificates.

3.3 Systems that call Identity

Connections into Identity from your own systems, such as partner applications and REST or web service integrations, depend on the caller, not on Identity. The calling system checks Identity's certificate, so its trust store is what needs the fix. Patching Identity's JDK does not change these connections.

This is a likely area of exposure. Integrations often use their runtime's default trust store (Java, Node.js, Python and others), and this applies to plain TLS and mutual TLS alike. Review these systems with their owners or vendors. Callers that pin Identity's certificate, or trust only a private or organizational CA, are not affected.

3.4 Affected versions

All Rhapsody Identity versions are potentially affected, because the fix belongs in the JDK, not in the Identity software. Identity does not bundle a Java runtime. Each version runs on a separately installed JDK (Amazon Corretto or Oracle JDK):

Identity version	Java version	Blocked-list file
v9.x to v11.x	Java 8	`<java home>/jre/lib/security/blacklisted.certs`
v12.x	Java 17	`<java home>/lib/security/blocked.certs`
v13.x	Java 21	`<java home>/lib/security/blocked.certs`

JDK builds released before October 23, 2026 do not block the compromised certificates. Builds released on or after that date should, and section 5.1 lists the first fixed build for each vendor and Java version.

Step 1 in section 5.2 can also be used to confirm whether a given installation is already protected. If the file named in the table is not present, check for the other name: older Java builds use `blacklisted.certs` and newer ones use `blocked.certs`.

SECTION 04

What Rhapsody is doing

- Applying the JDK update to all Rhapsody-hosted Identity environments, with restarts coordinated through scheduled maintenance windows.
- Publishing a supported procedure that customer-hosted sites can apply now (section 5).
- Including a check to verify the JDK update in Rhapsody procedures for new Identity installations and upgrades.
- Monitoring JDK vendor updates and revising this advisory if the risk picture changes.

SECTION 05

What you can do now

Updating the JDK blocks the compromised certificates and closes the risk for Identity.

Rhapsody Cloud customers	No action is needed. Rhapsody applies the update to hosted environments.
Customer-hosted customers	Apply the JDK update to every Identity environment, based on your own risk tolerance and change windows.
All customers	Separately from the Identity JDK, review the systems that call Identity and your other TLS-using applications with their owners or vendors (section 3.3).

How the fix works. The JDK update adds the compromised certificates to the JDK's blocked list, and also removes affected eMudhra roots and the legacy Sectigo "AAA Certificate Services" root from the default trust store. Certificates from DigiCert, Let's Encrypt and other reputable CAs continue to work.

One possible side effect. In the unlikely case that a remote endpoint's certificate chains to the AAA Certificate Services root, that connection will stop validating after the update. Contact the endpoint's owner about a certificate from a current root; as a stopgap, that root can be manually re-added to the new JDK's `cacerts`. This is safe on a fixed build because the known compromised intermediates remain blocked, but the workaround is lost at the next JDK update.

5.1 Install and configure the JDK update

Download the JDK update from your distribution vendor's website:

Identity version	Java version	Amazon Corretto: first fixed build	Oracle JDK: first fixed build
v13.x	21	21.0.13.0.11 (26 Oct 2026)	21.0.13.0.3 (24 Oct 2026)
v12.x	17	17.0.21.0.11 (26 Oct 2026)	17.0.21.0.3 (24 Oct 2026)
v9.x to v11.x	8	8.512.09.1 (26 Oct 2026)	8u511-b33 (24 Oct 2026)

- Any later build of the same line also contains the fix.
- Oracle downloads require a Java SE subscription.
- **Amazon Corretto:** see aws.amazon.com/security/security-bulletins/2026-150-aws for more information, and visit downloads.corretto.aws to download the latest update for your JDK version.
- **Oracle JDK:** see oracle.com/security-alerts/alert-cve-2026-114820.html for more information.

Once installed, ensure `JAVA_HOME` is set to the new update:

- If Identity runs as a service, check `set.default.JAVA_HOME` in `nextgate/products/ngs/etc/ngs-wrapper.conf`.
- If Identity runs as an application, check `JAVA_HOME` in `nextgate/products/ngs/bin/setenv` (Linux) or `setenv.bat` (Windows).

These settings override the server's `JAVA_HOME` environment variable; if `JAVA_HOME` is set, update it to the new JDK as well.

If certificates were imported into the old JDK's `cacerts` (for example, a directory server's certificate for LDAPS; see section 3.1), import them into the new JDK's `cacerts` before switching, or those connections will fail after the restart.

Once configured, restart the Identity service or application. The new JDK and associated block list will not be used until the software is restarted. Stop and start the Identity service, for example:

```
WINDOWS
net stop <ngs-service-name>
net start <ngs-service-name>
LINUX
sudo systemctl stop <ngs-service-name>
sudo systemctl start <ngs-service-name>
```

5.2 Validate the fix

1. **Check the blocked list** (works on every Java version). Open `<java home>\lib\security\blocked.certs` (on Java 8: `<java home>\jre\lib\security\blacklisted.certs`) and use Find to search for the two fingerprints in Appendix A. Both present means the JDK has the fix. Close the file without saving.
2. **Run the test program.** A test program by Mark Struberg, published with source in Guillaume Laforge's write-up, connects to the 12 public test endpoints. Save the source as `CheckCompromisedCa.java`, review it (it is short), and run it with the same JDK Identity uses:

   ```
   "<java home>\bin\java" CheckCompromisedCa.java
   ```

Before the fix, the program reports VULNERABLE 12 times. After the fix, each check reports OK; the logged failure reads either “Untrusted certificate” for eMudhra Inc, or “unable to find valid certification path to requested target.” Both mean the connection was correctly rejected. If it still reports VULNERABLE, check that you gave the test the full path to the new JDK, and that Identity’s `JAVA_HOME` and service configuration (section 5.1) point to the same one.

The test connects to the public test site `*.we-have-a-problem.com`, so it needs outbound HTTPS access. If the server has no internet access, test a copy of the same JDK on a machine that does. The test program requires Java 11 or later, so it cannot run on the Java 8 JDK used by Identity v11.x and earlier; for those installations, rely on the check in step 1, and contact Rhapsody Support if you need help validating.

The test starts its own JVM, so it sees the new JDK immediately; Identity itself is not protected until it has been restarted (section 5.1).

3. **Confirm Identity’s connections still work**, including notification delivery, directory (LDAP) logins and single sign-on. If a connection fails after the update, check whether that endpoint’s certificate chains to the AAA Certificate Services root (see “How the fix works” in section 5).

WHAT NOT TO DO

Do not disable certificate verification to work around TLS errors. That removes protection against exactly the attack this advisory addresses.

Do not hand-edit root certificates out of `cacerts` either. Removing only the emSign root leaves the Sectigo-signed path open, removal does nothing for connections that use Identity’s own trust stores, and manual edits are silently lost at the next JDK update.

The vendor JDK update is the supported fix: it blocks the specific compromised certificates on every connection the JVM makes, and removes the affected roots in a tested, controlled way.

SECTION 06

References

- eMudhra incident disclosure and test hosts: https://emudhra.we-have-a-problem.com/
- Mozilla incident report: https://bugzilla.mozilla.org/show_bug.cgi?id=2083449
- Chrome Root Program statement: https://blog.google/security/protecting-chrome-and-android-users-from-compromised-emudhra-intermediate-ca-certificates/
- Java CVE records (OpenJDK CVE-2026-91403 and Oracle CVE-2026-114820): https://www.cve.org/CVERecord?id=CVE-2026-91403, https://www.oracle.com/security-alerts/alert-cve-2026-114820.html
- JVM test program and fix walkthrough (Guillaume Laforge; includes the source of Mark Struberg's CheckCompromisedCa used in Validate the fix): https://glaforge.dev/posts/2026/10/21/the-emudhra-ca-compromise-why-your-jvm-is-vulnerable-and-how-to-fix-it/
- Azul: blocking the compromised certificates via `blocked.certs` (source of the blocked-certificates file in Appendix A): https://docs.azul.com/core/emudhra-workaround
- OpenJDK project guidance on the eMudhra CA compromise: https://openjdk.org/emudhra (the root removal it recommends is included in the fixed builds in section 5.1; see What not to do before making manual changes).
- Go security fix (CVE-2026-111943), for context only: https://access.redhat.com/security/cve/cve-2026-111943

APPENDIX A

Certificate fingerprints

Certificate names and keystore aliases do not uniquely identify a certificate; the SHA-256 fingerprint does. This appendix lists the fingerprints of the compromised eMudhra certificates, for identifying them and for confirming that a JDK blocks them.

These are intermediate certificates: they are presented by a server during the TLS handshake, not normally stored in your trust stores. The roots to look for in a trust store are the ones named in section 3.1 (`emSign ECC Root CA - G3` and `AAA Certificate Services`).

A.1 Quick check (used in section 5.2, Validate the fix)

A fixed JDK's blocked list contains all 48 fingerprints in A.2. To confirm an installation quickly, open `<java home>/lib/security/blocked.certs` (on Java 8: `<java home>/jre/lib/security/blacklisted.certs`) and use Find to search for these two fingerprints, one for each signing root. Close the file without saving.

```
# EM DV TLS CA - G3B (issued by emSign Root TLS CA - G3)
42C4ACCD12C4F59597925E1A08A7A90102608543809308D50EC427E118BA1493

# EM DV TLS CA - G3B (issued by AAA Certificate Services)
45A883F349AEDFD5F3A649B5498FFB121712BA8B448F3F2A91DBC5A766FCDDBE
```

If both are present, the installation blocks all of the compromised certificates. If either is missing, the JDK needs the update in section 5.

A.2 Full fingerprint list

The complete block list contains 48 SHA-256 fingerprints, covering the 16 certificates described in this advisory (the eight compromised intermediates, each signed by both the emSign root and the Sectigo AAA Certificate Services root) together with the related certificates revoked in the same incident. The same list ships in the fixed JDK builds (section 5.1) and in Azul's published emudhra-block-unix.certs file.

```
# eMudhra compromised intermediates (Oct 2026) - CVE-2026-91403
# 48 fingerprints, from Azul's emudhra-block-unix.certs
0164D7CEAE5BB909C272AF151FF44B1177AE9B47D9424B94F1D9558C30934D19
10E7E5BFDB55CFB78D0C21366654046182087A1960C079682513795665AA0C6B
169A0523C5E071AA4E298286DBD4EE0E40C8C06ABB6B0C077D132F001FA19240
21182E577E5106B669322ABE6086FFF6B101E8C83FD2D01DC32D854961AAC832
33C1BCDC6DA52DF950B3B14121127DA1955F7F31F828497A7608986EB2DDCC09
37E866582D43C35D7C17D47794B974F291B3047B868DC123382477F7A08AEFA8
3B88E4B77FC53C45CA216700E39FC0EA7621A0A40FCB268215082E35D2BD5829
3C5456D4219BA2372273867FAC9A07104600391E96C7FDCDE3B4420C0F1F69BF
42C4ACCD12C4F59597925E1A08A7A90102608543809308D50EC427E118BA1493
432EC25F0D28DE9006D97BF752F17BBB2CD41F669933FCB072D9A1148D0261F3
45A883F349AEDFD5F3A649B5498FFB121712BA8B448F3F2A91DBC5A766FCDDBE
4B2460A466F95FD40113BCDCCA2CEB29B3B093921DE45474DA7DF48874C630B1
4DED6F955374E40B8558A611A9719B2AB69AA6852AC50966EDC4B07DE4D4277C
50E71B08B9AC407B93D50EE90100CB6B6B41C956DC7AA86C5BAA38AE713CFD6A
558EC6CA13DA9B800FEDC9ACC3C18CA98BB19768F1309B60B5BB89793B3481CB
579480AEA166290BD3C643589E795FC0D0E54E86505D180F1DFF99EFD3199512
59CC5FD35A737B84682EDA26AD23F5ADE8AA847D5BEB3B38AB6CD62347925A66
5D8FC3ECDDB4DDB243AEE5C306BCFDE670ADF2D66B193DD3AA2D6B2FB91869BE
5F1DED4DD590AF74631290CF073B122A2AAC6A1FA479778A0BD43579BD832699
62B7F9868263F72F6911FC78CEAF72C1979158B7368BF648683BB2AA39B02E72
7485B51C0ED192878FA1D2F948FB529AE1842CC5086C4B67F7BE5B2039DE9489
75D8853BE99C87678E6C7BF9F684B31454D81DEB1C7F37B9FBC026A268337536
7BA62FEC54E9D040556FCA70A6A88F03761C178FA1F7CB8390A73076849F55A2
84112972B8E59D14560AE7E7119654474AAD119990CF3722BFC1E3B91615E1FD
8F580C6E34CEE7EE57B93BA9C0AC5FAEDD7BF3085F5EE9C6C0175A2555B111D0
8FED925362F62FFF90DD835BE6ADC32B02DEC8DF5E82D83E512F1C977D066E82
912A48745BFF2F14D1798B27858613DE1C0EB03C2BDE630AFD0DDD5251E1EBE1
98B896625C0C230ADEEC3EB2CF6AAD5001ECF9156A68644E82374CE5833BC25B
9E39DE2881273ACC225A149C72BA3E0F51424963669AB633EE1DC7DF5DB067B4
9E7617035CFF537C837E078A7CDC7FD9EB77BCE7DDF3DC6DBD5C50AAFC1F1FA7
A0ED2032C5249EE1A653F1F88005D9E779E385DF533B5C521137CEE5176CD7FC
A104B1314DF8E02F7DE27DDC287D0961D6E6824AD4DAFC3D11064F24C1B653D5
A4328E058C867AF9CA8D468C388B72C250CE84BDB6BAF7F8C47C9A0B1CB35EEC
A496E23AC0CAB8155FC9B117B97EAE368EFCBB9041AEC8804D5A308D2D01C2D8
A4B72DBD37256324B557B9176F8CBA4310DDFFBCC4609977E2463F3504A716C8
A61FE79B019062DD3C6420BC20EAF64702CC1063C546A084DD0A0BCB915A14A9
AB7292F82267333031F512379FDBADA949DFA49D3C046CC8FA7F34117C342986
B7121039CAF13E03E52507F42DB56FA802FF119DA7591ABA8A2E842133DB7C9C
B75104427B6A5BE55033AF50DC0FE3102181BE130E761D487A4E8178C93317DD
C5F1531C6331659C75A6411A088A65DAD7518FD91F57B64537A651B6DCD76AE0
C65CBC98295549BA940494A6214B9706E7115E64F3FE269412C5B48AEACB2222
C93F7B1EEF27CFD0F227DFE3F057D63833533890B7230B1821A424BD40848E33
CB6E15165AC802E9B3F993F16C25D41F4E81C21EBDE71AE1B012AC00BF9C7C3F
CD740652E552D93EE64D7C9D1CC25228E466895889CEE25A11C962AB09906978
D8A50FA0068E9558E6C3BEE18CDB03B7EECBC3C56C1B2336B98AEE11644994B8
E7568FE6E12331F4FD7DE61DA4C7559D0F8EB192A218D4741A3BA75184333838
EF34030C01B13BF42B534072DCC3C75B13C009DC9241119ADA39E2C15E2AFD2E
FD9D5C2DBBD4995AB69B8D50C439D9527B00529199C2FD01B2174EB58F07021F
```